

CDN (Content Delivery Network)

A CDN is a geographically distributed network of servers that delivers web content to users from locations closest to them, improving speed and reliability.

Key phrase **interviewers** love: “CDN serves content from the closest edge location instead of the origin.”

How It Works

Instead of all users requesting content from a single origin server, the CDN:

1. **Caches content** on edge servers in multiple geographic locations
2. **Routes requests** to the nearest edge server based on user location
3. **Serves cached content** directly from the edge
4. **Falls back** to the origin server only for cache misses or uncached content

Why do we need a CDN?

Imagine:

- Your backend is in **Toronto**
- Users are in **India, Europe, Australia**

Without CDN:

- Every request travels thousands of km
- High latency
- Backend gets hammered

With CDN:

- Content served from **edge nodes near users**
- Faster response
- Backend protected

Problems CDN solves

Problem	How CDN helps
High latency	Serve from nearest edge
Backend overload	Cache at edge
Traffic spikes	CDN absorbs traffic
DDoS	Edge absorbs attack
Bandwidth cost	Fewer origin hits

Key Benefits

Faster Load Times: Users download from nearby servers, reducing latency

Reduced Origin Load: The origin server handles fewer requests since most are served from cache, allowing it to scale more efficiently.

Improved Availability: If one server fails, requests route to another. Content remains accessible even during traffic spikes or partial outages.

Better Global Reach: Makes websites fast worldwide without deploying infrastructure in every region.

DDoS Protection: Distributed architecture absorbs malicious traffic across many servers, protecting the origin.

What Gets Cached

Static Assets: Images, CSS, JavaScript, fonts, videos—anything that doesn't change per user.

Dynamic Content: Some CDNs can cache personalized or frequently changing content with smart invalidation strategies.

API Responses: Cacheable API data can be served from the edge for faster response times.

Streaming Media: Video and audio content benefits greatly from CDN distribution.

Interview tip: “CDNs can cache dynamic content using TTLs, headers, and key-based caching.”

4 How CDN works (request flow)

Let's walk through **step-by-step** — this is gold in interviews.

Request Flow

1. User requests `https://example.com/image.png`
2. DNS resolves to **nearest CDN edge**
3. CDN checks cache:
 -  **Cache HIT** → return immediately
 -  **Cache MISS** → fetch from **Origin server**
4. CDN stores response
5. Returns response to user

SCSS

User → DNS → CDN Edge → (Cache?)
→ Origin Server (if miss)

👉 Use words: **Cache Hit, Cache Miss, Origin Fetch**

CDN Caching Strategies

Cache-Control Headers: Origin server tells the CDN how long to cache content (Cache-Control: max-age=3600).

Interview line: “We control CDN behavior using Cache-Control and HTTP headers.”

Purging/Invalidation: Manually clear cached content when it changes, either by URL or using cache tags.

Edge Computing: Some CDNs allow running code at the edge for dynamic content generation or API responses.

Popular CDN Providers

- **Cloudflare:** Free tier available, global network, security features
- **AWS CloudFront:** Integrates with AWS services, pay-as-you-go
- **Akamai:** Enterprise-focused, massive global presence
- **Fastly:** Edge computing capabilities, real-time purging
- **Google Cloud CDN:** Integrates with Google Cloud Platform
- **Azure CDN:** Microsoft's offering, multiple provider options

Common Use Cases

Website Performance: Serving static assets (images, CSS, JS) for faster page loads.

Video Streaming: Delivering video content without buffering to global audiences.

Software Distribution: Downloads of applications, updates, or large files.

API Acceleration: Caching API responses at the edge for reduced latency.

Gaming: Delivering game assets and updates to players worldwide.

Key Metrics

Cache Hit Ratio: Percentage of requests served from cache vs. origin—higher is better (typically aim for 80%+).

Time to First Byte (TTFB): How quickly the first byte reaches the user—CDNs dramatically reduce this.

Bandwidth Savings: Reduction in origin server bandwidth usage.

Geographic Coverage: Number and location of edge servers (Points of Presence or PoPs).

Configuration Considerations

Cache Duration: Balance between freshness and performance—longer caching reduces origin load but may serve stale content.

Cache Keys: What makes content unique? URLs, query parameters, cookies, headers can all affect caching.

Origin Shield: An additional caching layer between edge and origin to further reduce origin requests.

Geographic Restrictions: Some CDNs allow blocking or allowing content by country/region.